

Unsupervised Learning: Placement and Project Guide

Unsupervised learning finds structure when target labels are absent. Scale numeric features inside a **Pipeline**; select and validate hyperparameters without looking at a hidden test set. Internal scores are clues, not ground truth: combine them with stability checks and domain review.

K-Means Clustering

1. Overview

K-Means partitions observations into **k** compact clusters by minimizing within-cluster squared distance. It is a fast baseline for customer segments, image color compression, document grouping, and feature engineering.

2. Intuition

Put **k** movable pins among points. Repeatedly assign each point to its nearest pin, then move each pin to the average of its assigned points, until neither changes much.

3. Prerequisites

Vectors, Euclidean distance, mean, variance, feature scaling, local optima, and train/test leakage.

4. Core Concepts

- **Centroid**: a cluster's feature-wise mean; it represents a spherical, mean-like group. Interview: a centroid need not be an actual data point.
- **Inertia**: total squared point-to-centroid distance; lower is better for a fixed **k**, but always falls as **k** grows.
- **Initialization**: poor initial centroids can settle in a bad local optimum; **k-means++** spreads initial choices.

5. Algorithm / Working Process

Input: an **n × d** numeric matrix and **k**. Initialize centroids, assign every row to its closest centroid, recompute means, and repeat. Output: cluster labels, centroids, and inertia; inference assigns a new row to its nearest learned centroid.

6. Mathematical Foundation

It minimizes $J = \sum_i ||x_i - \mu_{(z_i)}||^2$, where z_i is a cluster assignment and μ_j is centroid j . Assignment minimizes J with fixed centroids; the arithmetic mean minimizes J with fixed assignments. Alternation converges to a local, not necessarily global, minimum.

7. Practical Implementation

```
from sklearn.datasets import load_wine
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans

X, _ = load_wine(return_X_y=True)
model = make_pipeline(StandardScaler(), KMeans(n_clusters=3, n_init=20,
random_state=42))
labels = model.fit_predict(X)
print(model[-1].inertia_, labels[:5])
```

8. Code Explanation

The scaler prevents large-unit features from dominating distance. `n_init=20` retains the best of several starts; `fit_predict` learns centroids and returns assignments.

9. Training / Evaluation

Choose `k` with an elbow plot, silhouette score, cluster stability across seeds/samples, and business usefulness. If labels are available only for audit, use ARI/NMI after clustering; do not tune on an evaluation label set.

10. Complexity and Cost

About $O(n*k*d*i)$ time for `i` iterations and $O(n*d + k*d)$ memory. CPU is sufficient for typical tabular data; MiniBatchKMeans suits millions of rows.

11. Common Use Cases

RFM customer segmentation, palette reduction, behavior cohorts, vector quantization, and coarse document clusters.

12. Common Mistakes

Skipping scaling; choosing `k` solely by the elbow; interpreting numeric IDs as distance features; ignoring outliers; assuming clusters are ground-truth classes.

13. Edge Cases / Limitations

It favors similarly sized, roughly spherical, equally dense clusters. Outliers pull means, and Euclidean distance weakens in high dimensions.

14. Variations

- **MiniBatchKMeans:** small random batches; use for scale, placement-relevant.
- **K-Medoids:** uses real representative points and is more robust to outliers.
- **Spherical K-Means:** cosine-oriented clustering for normalized text embeddings.

15. Related Topics

GMM softens K-Means assignments and models covariance. DBSCAN finds irregular density regions; PCA is often used before distance-based clustering.

16. Interview Questions

Question	Answer
What does K-Means optimize?	Within-cluster sum of squared distances.
Why is scaling needed?	Distance is unit-sensitive.
Why use <code>k-means++</code> ?	It usually gives better, faster-converging starts.
Is the result unique?	No; initialization can change a local optimum.
Why does inertia fall with k ?	Extra centroids can only reduce nearest-centroid distance.
How choose k ?	Elbow, silhouette, stability, and domain actionability.
Mean or median centroid?	Mean minimizes squared Euclidean distance.
Can it find moons?	Usually not; its clusters are convex/Voronoi-like.
How handle outliers?	Clean/cap them or use a robust alternative.
Training versus inference?	Iterative fitting versus one nearest-centroid lookup.

17. Practice Tasks

Implement one assignment/update iteration in NumPy; compare raw versus scaled Wine data; plot inertia/silhouette for `k=2..10`; test seed stability; explain three centroid profiles.

18. Project Ideas

- **Retail segmenter:** RFM features, pandas/scikit-learn, Online Retail data; demonstrates business interpretation.
- **Image palette compressor:** Pillow/OpenCV + K-Means, user images; demonstrates unsupervised CV.
- **Embedding explorer:** sentence-transformers + K-Means, news text; demonstrates semantic grouping.

19. Quick Revision

Key idea: alternate nearest-centroid assignment and means. Formula: SSE/inertia. Use for compact numeric groups. Metric: silhouette plus stability. Trap: unscaled features. One-liner: fast hard clustering around means.

20. Final Cheat Sheet

Definition: centroid partitioning. Input/output: vectors $\rightarrow$ labels and centroids. Steps: initialize, assign, average. Hyperparameters: `k`, `n_init`, `max_iter`. Pros: simple/fast; cons: shape and outlier sensitive. Best: segmentation baseline.

Hierarchical Clustering

1. Overview

Hierarchical clustering creates a nested tree of groups rather than requiring one final partition. It is useful when analysts want clusters at several granularities, such as gene families or product taxonomies.

2. Intuition

Agglomerative clustering begins with every point alone and repeatedly merges the most similar groups. A dendrogram records all merges, so a horizontal cut selects the desired number of clusters.

3. Prerequisites

Distance matrices, linkage, Euclidean/cosine distance, scaling, trees, and $O(n^2)$ memory intuition.

4. Core Concepts

- **Dendrogram:** merge tree; cut height controls granularity. Interview: merge height reflects linkage distance, not probability.
- **Linkage:** single uses nearest members, complete farthest, average mean pair distance, Ward variance increase.
- **Agglomerative versus divisive:** bottom-up is common; top-down recursively splits a whole cluster.

5. Algorithm / Working Process

Input points or a precomputed distance matrix. Start singleton clusters, repeatedly merge the pair selected by linkage, then cut the tree at `n_clusters` or `distance_threshold`. Output labels and optionally a dendrogram; new-point inference is not naturally defined.

6. Mathematical Foundation

For clusters A, B , single linkage is $\min d(a, b)$, complete is $\max d(a, b)$, and average is mean pairwise distance. Ward chooses the merge with smallest increase in SSE, approximately $|A||B|/(|A|+|B|) * ||\mu_A - \mu_B||^2$.

7. Practical Implementation

```
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import AgglomerativeClustering

X, _ = load_wine(return_X_y=True)
X = StandardScaler().fit_transform(X)
labels = AgglomerativeClustering(n_clusters=3, linkage="ward").fit_predict(X)
print(labels[:5])
```

8. Code Explanation

Ward linkage requires Euclidean features and tends to make compact groups. Scaling happens before clustering because every merge depends on the chosen distances.

9. Training / Evaluation

Inspect a dendrogram before committing to a cut; compare linkage/distance choices and bootstrap stability. Silhouette is useful after choosing labels, but domain interpretability should decide the final cut.

10. Complexity and Cost

Naive implementations are costly; typical agglomerative workflows need $O(n^2)$ memory and about $O(n^2 \log n)$ to $O(n^3)$ time. It is not a first choice for huge data.

11. Common Use Cases

Taxonomies, gene-expression groups, document/topic exploration, customer subsegments, and visual cluster reports.

12. Common Mistakes

Using single linkage without checking chaining; applying Ward to non-Euclidean distances; reading dendrogram heights as confidence; forcing a cut without a decision use case.

13. Edge Cases / Limitations

Early bad merges cannot be undone. Single linkage chains through bridges, complete linkage fragments elongated groups, and scale is limited by pairwise distances.

14. Variations

- **Single/complete/average linkage:** choose based on desired compactness; placement-essential.
- **Divisive clustering:** top-down splits; useful conceptually, less common in libraries.
- **BIRCH:** summarizes large datasets before hierarchical refinement.

15. Related Topics

K-Means returns one flat partition; DBSCAN treats density-connected groups as clusters. Spectral clustering changes the similarity space before clustering.

16. Interview Questions

Question	Answer
Why hierarchical clustering?	It exposes cluster structure at multiple resolutions.
What is a dendrogram?	A tree of merges/splits.
Single-linkage weakness?	Chaining through nearby bridges.
Complete-linkage behavior?	Favors compact groups.
What does Ward minimize?	Increase in within-cluster variance/SSE.

Question	Answer
Need k beforehand?	No; choose a later tree cut.
Can merges be undone?	Not in standard agglomerative clustering.
Why scale?	Linkage depends on distances.
Can it predict new points?	Not naturally; assign separately or refit.
Main scaling limit?	Pairwise distance memory.

17. Practice Tasks

Draw dendrograms for three linkage choices; find a chaining example; cut one tree at two heights; compare Ward with K-Means; test stability under resampling.

18. Project Ideas

- **Product taxonomy:** product embeddings + SciPy dendrogram; clear catalog-analytics story.
- **Gene cluster explorer:** scikit-learn + heatmaps, gene-expression data; research-oriented.
- **News hierarchy:** TF-IDF/SVD + agglomeration, 20 Newsgroups; demonstrates NLP exploration.

19. Quick Revision

Key idea: nested merges. Formula: linkage distance. Use when resolution is unknown. Metric: silhouette/stability. Trap: linkage choice. One-liner: clustering that preserves a merge tree.

20. Final Cheat Sheet

Definition: tree-based clustering. Input/output: points -> dendrogram/labels. Steps: distances, repeated merge, cut. Hyperparameters: linkage, metric, cut. Pros: interpretable hierarchy; cons: costly/irreversible. Best: exploratory taxonomy.

DBSCAN

1. Overview

DBSCAN (Density-Based Spatial Clustering of Applications with Noise) discovers arbitrary-shaped dense regions and labels sparse points as noise. It is useful for geospatial hotspots, fraud pre-screening, and noisy sensor data.

2. Intuition

Imagine placing a circle of radius `eps` around every point. A point with at least `min_samples` neighbors is a dense core; connected core circles form a cluster, while isolated points are noise.

3. Prerequisites

Distance metrics, nearest neighbors, density, feature scaling, curse of dimensionality, and basic spatial indexes.

4. Core Concepts

- **Core point:** has at least `min_samples` points in its `eps` neighborhood; it expands a cluster.
- **Border point:** near a core but not dense itself; belongs to a cluster but cannot expand it.
- **Noise:** neither reachable from a core nor dense; scikit-learn labels it `-1`.

5. Algorithm / Working Process

Input points, `eps`, `min_samples`, and a metric. Find neighborhoods, start a cluster at each unvisited core, grow it through density-reachable cores, attach border points, and mark leftovers as noise. There is no standard predictive assignment for unseen data.

6. Mathematical Foundation

$N_{\text{eps}}(p) = \{q: d(p,q) \leq \text{eps}\}$. `p` is core when $|N_{\text{eps}}(p)| \geq \text{min_samples}$. A cluster is a maximal set connected by density reachability; this connection lets it trace non-convex shapes.

7. Practical Implementation

```
from sklearn.datasets import make_moons
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import DBSCAN

X, _ = make_moons(n_samples=500, noise=.08, random_state=42)
labels = make_pipeline(StandardScaler(), DBSCAN(eps=.22,
min_samples=8)).fit_predict(X)
print("noise points:", (labels == -1).sum())
```

8. Code Explanation

`make_moons` tests a shape K-Means struggles with. Scaling makes `eps` meaningful; `-1` indicates points considered noise, not a real cluster.

9. Training / Evaluation

Use a k-distance plot (distance to the `min_samples`-th neighbor) to find an `eps` knee, then inspect noise rate and cluster stability. Validate with geospatial/business labels only after tuning is frozen.

10. Complexity and Cost

With an efficient spatial index, often near $O(n \log n)$ in low dimensions; brute force is $O(n^2)$. Neighborhood storage can be large, and high dimensions destroy efficient density contrast.

11. Common Use Cases

GPS pickup hotspots, duplicate/near-duplicate detection, anomaly candidates, astronomy point clouds, and irregular image regions.

12. Common Mistakes

Not scaling; treating all **-1** points as errors; choosing one global **eps** for varying densities; using Euclidean distance blindly for text embeddings; evaluating only non-noise samples.

13. Edge Cases / Limitations

One **eps** cannot fit both sparse and dense clusters. It can merge groups connected by a dense bridge and is unreliable in high-dimensional sparse spaces.

14. Variations

- **HDBSCAN**: variable-density hierarchy; excellent practical extension.
- **OPTICS**: density ordering that avoids committing to one **eps**.
- **ST-DBSCAN**: adds spatial-temporal distance constraints.

15. Related Topics

Unlike K-Means/GMM, DBSCAN does not require **k** and explicitly finds noise. Isolation Forest is an anomaly method rather than a clusterer.

16. Interview Questions

Question	Answer
What are its two key parameters?	Neighborhood radius eps and density threshold min_samples .
What is a core point?	A point with enough neighbors inside eps .
What label means noise?	-1 in scikit-learn.
Does it need k?	No.
Can it find moons?	Yes, if density is appropriate.
Why scale?	eps is a distance threshold.
Main weakness?	Varying-density/high-dimensional data.
What is a border point?	Non-core point reachable from a core.
How choose eps?	k-distance plot plus stability/domain review.
DBSCAN vs K-Means?	Density/noise/arbitrary shapes versus centroid partitions.

17. Practice Tasks

Plot core/border/noise points; tune **eps** from a k-distance graph; compare moons with K-Means; test Manhattan versus Euclidean distance; diagnose a cluster-merging bridge.

18. Project Ideas

- **Taxi hotspot finder**: geopandas/scikit-learn, NYC Taxi sample; applied geospatial clustering.

- **Transaction anomaly screen:** DBSCAN + dashboard, credit-card features; demonstrates noise handling.
- **Store co-location map:** OpenStreetMap POIs + DBSCAN; useful location-intelligence portfolio item.

19. Quick Revision

Key idea: density-connected cores grow clusters. Formula: $\text{eps-neighborhood/core condition}$. Use for noisy irregular groups. Metric: stability/noise rate. Trap: one density assumption. One-liner: density clustering with explicit outliers.

20. Final Cheat Sheet

Definition: density-based clustering. Input/output: points $\rightarrow$ labels/noise. Steps: neighborhoods, cores, expansion. Hyperparameters: `eps`, `min_samples`, metric. Pros: shapes/noise/no k ; cons: density-sensitive. Best: spatial/noisy data.

Principal Component Analysis (PCA)

1. Overview

PCA is a linear dimensionality-reduction method that rotates correlated features into orthogonal components ordered by explained variance. It supports visualization, compression, denoising, and preprocessing for distance-based models.

2. Intuition

View a tilted cloud of 2-D points from its longest direction: one coordinate captures most variation and the short direction may be discarded with limited reconstruction loss.

3. Prerequisites

Means/centering, covariance, variance, dot products, eigenvectors or SVD, matrix shapes, and scaling.

4. Core Concepts

- **Principal component:** a unit direction of maximum remaining variance; components are orthogonal.
- **Explained variance ratio:** fraction of total variance retained by a component; use cumulative ratio to select dimensionality.
- **Loadings:** feature weights in a component; large magnitude indicates contribution, not causality.

5. Algorithm / Working Process

Input a centered numeric matrix. Standardize when units differ, compute SVD/covariance eigendirections, retain top r directions, and project $Z = X_{\text{centered}} W_r$. Output lower-dimensional coordinates; inverse transform approximates original features.

6. Mathematical Foundation

For centered X , covariance is $C = X^T X / (n-1)$. Eigenvectors of C are directions and eigenvalues are component variances. PCA minimizes rank- r reconstruction error $\|X - X W_r W_r^T\|_F^2$; SVD gives $X = U \Sigma V^T$.

7. Practical Implementation

```
from sklearn.datasets import load_wine
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X, _ = load_wine(return_X_y=True)
pca = make_pipeline(StandardScaler(), PCA(n_components=.95, random_state=42))
Z = pca.fit_transform(X)
print("components kept:", Z.shape[1])
```

8. Code Explanation

`StandardScaler` is essential when variables use different units. `n_components=.95` selects the minimum number retaining 95% variance; `Z` is the compressed representation.

9. Training / Evaluation

Fit PCA only on training folds, then transform validation/test data. Evaluate downstream metrics, reconstruction error, explained variance, and visualization stability; maximizing variance does not guarantee preserving the label signal.

10. Complexity and Cost

Full SVD is roughly $O(\min(n*d^2, d*n^2))$; randomized/incremental solvers help at scale. Transforming a row costs $O(d*r)$ and memory is about $O(d*r)$ after fitting.

11. Common Use Cases

2-D inspection, correlated-feature compression, image eigenfaces, denoising, and preprocessing before K-Means/KNN.

12. Common Mistakes

Skipping centering; scaling binary/meaningful-unit features uncritically; fitting before split; treating PCs as causal factors; using only explained variance instead of task performance.

13. Edge Cases / Limitations

PCA captures linear, high-variance structure; low-variance features can be predictive. It is outlier-sensitive and components may be hard to explain.

14. Variations

- **Kernel PCA:** nonlinear projection; useful for curved manifolds.
- **Incremental/Randomized PCA:** scalable approximation; project-relevant.
- **Sparse PCA:** sparse loadings for interpretability; research/analytics use.

15. Related Topics

SVD is its computational foundation. t-SNE/UMAP prioritize local visualization rather than variance; autoencoders learn nonlinear compression.

16. Interview Questions

Question	Answer
What does PCA maximize?	Projected variance, equivalently minimum linear reconstruction error.
Why center?	Otherwise the mean direction dominates covariance.
Why scale?	Variance depends on units.
Are PCs correlated?	No; they are orthogonal and uncorrelated on centered data.
How choose components?	Cumulative variance plus downstream validation.
PCA versus feature selection?	PCA creates features; selection retains original ones.
Does PCA use labels?	No.
Is it robust to outliers?	No, use robust preprocessing/alternatives.
Can it be nonlinear?	Kernel PCA can.
Why before K-Means?	It can denoise/reduce dimensions, but validate.

17. Practice Tasks

Derive PCA with NumPy SVD; plot cumulative variance; reconstruct data at 2/5/10 PCs; compare clustering with and without PCA; inspect loadings.

18. Project Ideas

- **Wine visual analyzer:** PCA + Plotly, UCI Wine; interpretable EDA portfolio piece.
- **Face compressor:** PCA + OpenCV, LFW faces; shows reconstruction trade-offs.
- **Sensor denoiser:** PCA + anomaly checks, UCI HAR; connects reduction and operations data.

19. Quick Revision

Key idea: retain maximum linear variance in fewer axes. Formula: $Z = XW$. Use for compression/visualization. Metric: explained variance plus task score. Trap: fit scaler/PCA before split. One-liner: orthogonal variance-maximizing projection.

20. Final Cheat Sheet

Definition: linear orthogonal reduction. Input/output: features -> principal scores. Steps: center, SVD, retain, project. Hyperparameters: components/solver. Pros: fast/denoising; cons: linear/outlier-sensitive. Best: correlated numeric features.

t-SNE Intuition

1. Overview

t-distributed Stochastic Neighbor Embedding (t-SNE) is a nonlinear method for visualizing high-dimensional points in two or three dimensions. It is useful for inspecting embedding neighborhoods, class separation, and possible data-quality issues; it is not normally a production feature transform.

2. Intuition

In the original space, each point gives nearby points high friendship probabilities and distant points low probabilities. t-SNE places points on a map so these friendships match. The heavy-tailed Student-t distribution gives crowded distant points room to move apart.

3. Prerequisites

Euclidean distance, conditional probability, Gaussian distribution, KL divergence, gradient descent, scaling, and PCA.

4. Core Concepts

- **Perplexity:** an effective neighbor count; it controls local scale. Interview: it is not a cluster count.
- **KL divergence:** penalizes missing true neighbors strongly, preserving local neighborhoods.
- **Crowding problem:** a low-dimensional map has too little area for all moderate distances; the Student-t tail mitigates it.
- **Random seed:** layouts can rotate, flip, or differ; compare neighborhood structure, not coordinates.

5. Algorithm / Working Process

Input is a scaled feature matrix, often PCA-reduced first. Compute high-dimensional pair probabilities p_{ij} , initialize 2-D coordinates, compute low-dimensional q_{ij} , then optimize coordinates to make Q resemble P . Output is visualization coordinates; standard t-SNE has no reliable out-of-sample **transform**.

6. Mathematical Foundation

For a Gaussian bandwidth σ_i , $p_{j|i} = \exp(-||x_i - x_j||^2 / (2 \sigma_i^2)) / \sum_{k \neq i} \exp(-||x_i - x_k||^2 / (2 \sigma_i^2))$. Symmetrize as $p_{ij} = (p_{j|i} + p_{i|j}) / (2n)$. In the map, $q_{ij} = (1 + ||y_i - y_j||^2)^{-1} / \sum_{k \neq i} (1 + ||y_i - y_k||^2)^{-1}$. Minimize $KL(P||Q) = \sum_{ij} p_{ij} \log(p_{ij}/q_{ij})$ by gradient descent; early exaggeration temporarily multiplies P to separate groups.

7. Practical Implementation

```
from sklearn.datasets import load_digits
from sklearn.manifold import TSNE
from sklearn.preprocessing import StandardScaler

X, y = load_digits(return_X_y=True)
Z = TSNE(n_components=2, perplexity=30, learning_rate="auto",
        init="pca",
        random_state=42).fit_transform(StandardScaler().fit_transform(X))
print(Z.shape, y[:5]) # plot Z[:, 0], Z[:, 1], colored by y
```

8. Code Explanation

`StandardScaler` makes pixel-feature distances comparable. `init="pca"` is a stable initialization; `Z` is only a two-dimensional visualization, while `y` is used only afterward to inspect the plot.

9. Training / Evaluation

Use a representative sample; first reduce very high-dimensional data to roughly 30--50 PCs. Tune perplexity (commonly 5--50) and learning rate. Evaluate trustworthiness/neighborhood preservation and repeated seeds, not classification accuracy from visually separated blobs.

10. Complexity and Cost

Exact t-SNE has roughly $O(n^2)$ time and memory; Barnes-Hut approximations are about $O(n \log n)$ in 2-D. It is CPU-heavy for tens of thousands of points and primarily an offline visualization tool.

11. Common Use Cases

Embedding diagnostics for NLP/LLMs, single-cell biology, image-feature inspection, exploratory clustering, and mislabeled-example discovery.

12. Common Mistakes

Treating apparent clusters as proof; reading global distances or cluster sizes literally; fitting on unscaled data; using labels to tune the map; comparing maps with different seeds/hyperparameters as if axes match.

13. Edge Cases / Limitations

It can manufacture visually separated islands from continuous structure. Global geometry, density, and inter-cluster distance are unreliable. It is slow, stochastic, and lacks a principled standard transform for new rows.

14. Variations

- **Barnes-Hut t-SNE:** approximate repulsion; practical default for medium data.
- **FIt-SNE/openTSNE:** faster interpolation-based implementations; useful at large scale.
- **Parametric t-SNE:** neural-network mapping for new points; research/project extension.

15. Related Topics

PCA preserves global linear variance; UMAP is faster and supports transforms. Spectral clustering uses graph neighborhoods for clustering, whereas t-SNE uses them for a visualization.

16. Interview Questions

Question	Answer
What does t-SNE preserve?	Local neighborhood probabilities.
Why Student-t in 2-D?	Its heavy tail reduces crowding and enables separation.
What is perplexity?	An effective number of neighbors, not k .
Can map distances be trusted globally?	Generally no.
Why early exaggeration?	It helps local groups separate during early optimization.
Why use PCA first?	Denoising and lower cost.
Is t-SNE deterministic?	Only with controlled seed and settings; it remains sensitive.
Can it embed a new point?	Not directly in standard scikit-learn t-SNE.
Which loss is minimized?	$\text{KL}(P)$
t-SNE versus UMAP?	t-SNE emphasizes local visualization; UMAP is faster and transformable.

17. Practice Tasks

Plot Digits for perplexities 5, 30, and 50; calculate trustworthiness; repeat three seeds; compare raw pixels versus PCA(30)+t-SNE; identify points whose nearest visual neighbors disagree with labels.

18. Project Ideas

- **Embedding QA explorer:** sentence-transformers + t-SNE + Streamlit; use support-ticket text; strong LLM/RAG debugging story.
- **Digit error atlas:** scikit-learn + Plotly; Digits/Fashion-MNIST; highlights ambiguous/mislabeled cases.
- **Single-cell map notebook:** Scanpy + t-SNE; PBMC data; demonstrates biological exploratory analysis.

19. Quick Revision

Key idea: match local neighbor probabilities in 2-D. Formula: minimize $\text{KL}(P || Q)$. Use for visualization. Metric: trustworthiness. Trap: interpreting global map geometry. One-liner: a neighborhood-preserving, not distance-preserving, map.

20. Final Cheat Sheet

Definition: nonlinear local visualization. Input/output: vectors $\rightarrow$ 2-D/3-D points. Steps: affinities, initialize, optimize KL. Hyperparameters: perplexity, learning rate, iterations, seed. Pros: revealing local groups; cons: slow and globally misleading. Best: inspecting embeddings.

UMAP Intuition

1. Overview

Uniform Manifold Approximation and Projection (UMAP) is a nonlinear manifold-learning method for visualization and compact embeddings. It is widely used for large embedding datasets because it is comparatively fast and can transform new samples after fitting.

2. Intuition

Build a weighted nearest-neighbor graph: points connected strongly in the original space should stay connected on a smaller map. UMAP pulls graph neighbors together and pushes random non-neighbors apart, like arranging a social network on a page.

3. Prerequisites

k-nearest neighbors, graphs, distance metrics, probability-like weights, cross-entropy, stochastic optimization, PCA, and scaling.

4. Core Concepts

- **Fuzzy simplicial set:** a weighted neighbor graph encoding local connectivity; it is the central UMAP object.
- **n_neighbors:** local versus broader structure trade-off; small values emphasize local detail.
- **min_dist:** how tightly points may pack in the map; it changes visual compactness, not true class evidence.
- **Metric:** cosine is often better for normalized text embeddings; Euclidean is common for scaled tabular data.

5. Algorithm / Working Process

Input is vectors plus a metric. UMAP finds approximate nearest neighbors, converts neighborhoods into weighted edges, initializes low-dimensional coordinates, and samples attractive/repulsive updates. Output is coordinates and a fitted mapper that can approximately embed new vectors.

6. Mathematical Foundation

For neighbor j of i , UMAP uses a local membership strength approximately $w_{ij} = \exp(-(d(x_i, x_j) - \rho_i) / \sigma_i)$, where ρ_i corrects local connectivity and σ_i adapts scale. Low-dimensional similarity has form $v_{ij} = 1 / (1 + a ||y_i - y_j||^{(2b)})$. It minimizes fuzzy-set cross-entropy: $\sum_{ij} [w_{ij} \log(v_{ij}) + (1 - w_{ij}) \log(1 - v_{ij})]$, optimized with negative sampling.

7. Practical Implementation

```
# pip install umap-learn
from sklearn.datasets import load_digits
from sklearn.preprocessing import StandardScaler
import umap.umap_ as umap
```

```
X, y = load_digits(return_X_y=True)
mapper = umap.UMAP(n_neighbors=20, min_dist=0.1, metric="euclidean",
random_state=42)
Z = mapper.fit_transform(StandardScaler().fit_transform(X))
print(Z.shape, mapper.transform(X[:3]).shape)
```

8. Code Explanation

The fitted `mapper` stores the learned graph/layout and supports `transform` for new rows. `n_neighbors=20` balances local and mid-range structure; the output `Z` should be plotted, not treated as a calibrated coordinate system.

9. Training / Evaluation

Scale inputs and choose a metric compatible with the representation. Sweep `n_neighbors`, `min_dist`, and dimensionality; assess neighbor trustworthiness, seed stability, downstream validation when used as preprocessing, and domain inspection.

10. Complexity and Cost

Approximate neighbor search makes fitting near $O(n \log n)$ in common settings; memory is dominated by the neighbor graph, roughly $O(n * n_neighbors)$. CPU works well; GPU implementations help large datasets.

11. Common Use Cases

Text/image embedding exploration, single-cell analysis, interactive map dashboards, pre-clustering compression, and anomaly/outlier inspection.

12. Common Mistakes

Treating gaps as proof of groups; using Euclidean distance on unnormalized semantic embeddings; setting `min_dist=0` just to make pretty clusters; fitting on production test data; comparing absolute axes across runs.

13. Edge Cases / Limitations

Global distances and densities can be distorted; results depend on metric and neighbor choices. New-point transforms are approximate and may drift if the data distribution changes. It is not a replacement for supervised validation.

14. Variations

- **Supervised UMAP:** uses labels while fitting; useful for visualization, not unbiased evaluation.
- **densMAP:** aims to retain local density better; research/EDA use.
- **Parametric UMAP:** neural mapping for scalable inference; useful for deployment.

15. Related Topics

t-SNE also preserves local neighborhoods but is slower and commonly non-transformable. PCA provides a stable linear baseline. HDBSCAN is often applied to UMAP embeddings, but validate on original-space evidence too.

16. Interview Questions

Question	Answer
What structure does UMAP build first?	A weighted k-nearest-neighbor graph.
What does <code>n_neighbors</code> control?	Locality versus broader structure.
What does <code>min_dist</code> control?	Minimum packing distance in the low-dimensional map.
Can UMAP transform new data?	Yes, approximately, after fitting.
Why does metric matter?	It defines neighborhood meaning.
Is UMAP a clustering algorithm?	No; it is a dimension-reduction/visualization method.
Is its global geometry exact?	No.
Why use negative sampling?	Efficiently approximates repulsion from non-neighbors.
UMAP versus PCA?	Nonlinear local graph mapping versus linear variance projection.
When use cosine distance?	Often for normalized text or embedding vectors.

17. Practice Tasks

Compare `n_neighbors` 5/20/100; vary `min_dist`; use cosine versus Euclidean on sentence embeddings; evaluate `transform` on held-out rows; cluster before and after UMAP and compare stability.

18. Project Ideas

- **Semantic search map:** sentence-transformers + UMAP + Streamlit; FAQ corpus; useful RAG observability portfolio item.
- **Product catalog explorer:** CLIP embeddings + UMAP; public product images; demonstrates multimodal discovery.
- **Network telemetry explorer:** Pandas + UMAP; UNSW-NB15; demonstrates security-anomaly EDA.

19. Quick Revision

Key idea: preserve a fuzzy neighbor graph. Formula: graph cross-entropy. Use for fast nonlinear maps. Metric: trustworthiness/stability. Trap: over-reading clusters and distances. One-liner: graph-based local manifold approximation.

20. Final Cheat Sheet

Definition: nonlinear graph-based reduction. Input/output: vectors -> low-D coordinates. Steps: kNN graph, fuzzy weights, attractive/repulsive optimization. Hyperparameters: `n_neighbors`, `min_dist`, metric. Pros: fast/transformable; cons: distorted global geometry. Best: large embedding exploration.

Anomaly Detection

1. Overview

Anomaly detection identifies rare observations that differ meaningfully from normal behavior. It supports fraud screening, manufacturing monitoring, intrusion detection, medical triage, and data-quality checks. An anomaly score is a ranking signal; it is not automatically a diagnosis.

2. Intuition

Learn the normal pattern, then flag observations that do not fit it. A credit-card transaction far from a customer's usual amount, country, time, and merchant pattern receives a high unusualness score.

3. Prerequisites

Feature engineering, distributions, z-scores, distances, classification metrics under imbalance, time-series splits, and contamination/threshold concepts.

4. Core Concepts

- **Point/contextual/collective anomaly:** unusual row, unusual given context, or unusual sequence; the feature design must match the type.
- **Novelty versus outlier detection:** novelty trains on clean normal data; outlier detection allows contamination in training data.
- **Score and threshold:** rank with a continuous score, then select threshold by business cost/capacity.
- **Contamination:** expected anomalous fraction; a useful prior, not a truth.

5. Algorithm / Working Process

Input is usually unlabeled or mostly-normal feature vectors. Clean obvious corruption, fit a normality/density/isolation/reconstruction model, compute anomaly scores, choose a threshold on a validation period, and send highest-risk cases to review. At inference, score and monitor score drift plus reviewer outcomes.

6. Mathematical Foundation

A simple univariate score is $z=(x-\mu)/\sigma$; flag large $|z|$ only when a Gaussian assumption is reasonable. Multivariate Gaussian scoring uses Mahalanobis distance $D_M^2=(x-\mu)^T \Sigma^{-1}(x-\mu)$ or negative log likelihood. If labels exist, use precision, recall, PR-AUC, recall at review budget k , and cost: $C = c_{FP} FP + c_{FN} FN$ rather than raw accuracy.

7. Practical Implementation

```
from sklearn.ensemble import IsolationForest
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
import numpy as np
```

```
rng = np.random.default_rng(42)
X = np.r_[rng.normal(0, 1, (500, 2)), [[7, 7], [-6, 5]]]
model = make_pipeline(StandardScaler(), IsolationForest(contamination=0.01,
random_state=42))
labels = model.fit_predict(X)          # -1 anomaly, 1 normal
scores = -model.decision_function(X)   # larger = more anomalous
print(np.flatnonzero(labels == -1), scores.max())
```

8. Code Explanation

The pipeline prevents scale artifacts. `fit_predict` outputs a thresholded decision, while `decision_function` retains a continuous ranking score; keep scores for triage and tune the operating threshold separately.

9. Training / Evaluation

Use chronological splits for events, prevent entity leakage, and label a representative sample of alerts. Report PR-AUC, precision/recall at a fixed alert volume, detection latency, and financial/operational cost. Retrain only after investigating drift and feedback quality.

10. Complexity and Cost

Cost depends on method: z-score is $O(nd)$, nearest-neighbor methods can be expensive, tree methods are practical for tabular data, and deep autoencoders may need GPUs. Human review often dominates total cost.

11. Common Use Cases

Payment fraud, account takeover, sensor faults, unusual logs, rare medical measurements, content moderation, and ETL-quality monitoring.

12. Common Mistakes

Using accuracy on 0.1% positives; random splitting time data; calling every rare customer legitimate fraud; choosing threshold without review capacity; leaking future aggregates; treating unsupervised alerts as labels.

13. Edge Cases / Limitations

Rare does not mean bad; normal behavior can be multimodal and change over time. Unlabeled evaluation is difficult, adversaries adapt, and a global model can unfairly flag underrepresented groups.

14. Variations

- **Statistical control charts:** fast for stable univariate sensors; placement-relevant.
- **LOF/kNN:** local-density anomalies; works for varying-density data but is costly.
- **Isolation Forest:** isolates sparse points efficiently; strong tabular baseline.
- **Autoencoder:** high reconstruction error; suitable for nonlinear high-dimensional normal data.

15. Related Topics

One-class SVM and Isolation Forest are dedicated unsupervised detectors. GMM scores likelihood. Supervised fraud classifiers are preferable once reliable labels exist; change-point detection handles temporal regime

shifts.

16. Interview Questions

Question	Answer
Why is accuracy poor here?	A model predicting all normal can have high accuracy.
What is novelty detection?	Train on clean normals, detect unseen deviations.
How choose a threshold?	Validation cost, desired recall, and review capacity.
What is contamination?	Assumed anomaly proportion used by some methods.
Which split for transactions?	Time-based, often grouped by entity.
What metric is useful?	PR-AUC and precision/recall at alert budget.
Why keep a score?	It allows flexible, cost-aware thresholding.
Can unsupervised anomalies be fraud?	No; they require investigation/context.
What causes false positives?	Drift, rare valid subgroups, weak context features.
When use supervised learning?	When high-quality incident labels are available.

17. Practice Tasks

Implement z-score and IQR baselines; compare IF/LOF/One-Class SVM on a synthetic set; tune threshold under a 20-alert/day budget; simulate drift; audit false positives by customer segment.

18. Project Ideas

- **Fraud alert prioritizer:** scikit-learn + FastAPI; Credit Card Fraud data; shows imbalanced evaluation.
- **Sensor health monitor:** Pandas + Isolation Forest; NASA turbofan data; demonstrates time-aware validation.
- **Log anomaly dashboard:** sentence embeddings + UMAP/IF; HDFS logs; strong MLOps observability story.

19. Quick Revision

Key idea: score deviation from normal. Formula: z or $-\log p(x)$. Use for rare-risk triage. Metrics: PR-AUC and recall at budget. Trap: random splits/accuracy. One-liner: anomaly detection ranks unusual behavior, not verified bad behavior.

20. Final Cheat Sheet

Definition: identify deviations from a learned normal pattern. Input/output: features -> anomaly score/alert. Steps: engineer context, fit, score, threshold, review. Hyperparameters: contamination/threshold. Pros: works with few labels; cons: ambiguous evaluation/drift. Best: risk screening and monitoring.

Gaussian Mixture Models (GMM)

1. Overview

A Gaussian Mixture Model represents data as a weighted sum of Gaussian components. It provides soft clusters, ellipsoidal cluster shapes, density estimation, and likelihood-based anomaly scores.

2. Intuition

Imagine several translucent bell-shaped clouds. A point may partly belong to each cloud; its memberships are probabilities rather than a hard K-Means label.

3. Prerequisites

Multivariate Gaussian distribution, covariance matrices, likelihood, Bayes rule, logarithms, and K-Means.

4. Core Concepts

- **Component:** one Gaussian with mean and covariance; captures an elliptical region.
- **Mixture weight:** π_k , nonnegative and summing to one; represents component prevalence.
- **Responsibility:** posterior $r_{ik} = P(z_i=k | x_i)$; the soft assignment.
- **Covariance type:** full, tied, diagonal, spherical; it controls flexibility and parameter count.

5. Algorithm / Working Process

Input is continuous features and component count. Initialize means/covariances/weights, alternate E-step responsibilities with M-step weighted parameter updates, and stop when log likelihood stabilizes. Inference returns membership probabilities, labels, samples, or $\log p(x)$.

6. Mathematical Foundation

$p(x_i) = \sum_{k=1}^K \pi_k N(x_i | \mu_k, \Sigma_k)$. Responsibilities are $r_{ik} = \frac{\pi_k N(x_i | \mu_k, \Sigma_k)}{\sum_j \pi_j N(x_i | \mu_j, \Sigma_j)}$. EM maximizes $\sum_i \log p(x_i)$ with $N_k = \sum_i r_{ik}$, $\mu_k = (1/N_k) \sum_i r_{ik} x_i$, $\Sigma_k = (1/N_k) \sum_i r_{ik} (x_i - \mu_k)(x_i - \mu_k)^T$, and $\pi_k = N_k/n$.

7. Practical Implementation

```
from sklearn.datasets import load_wine
from sklearn.mixture import GaussianMixture
from sklearn.preprocessing import StandardScaler

X, _ = load_wine(return_X_y=True)
X = StandardScaler().fit_transform(X)
gmm = GaussianMixture(n_components=3, covariance_type="full", n_init=10,
random_state=42).fit(X)
print(gmm.predict_proba(X[:2]), gmm.bic(X), gmm.score_samples(X[:2]))
```

8. Code Explanation

`predict_proba` gives responsibilities, `score_samples` gives per-row log likelihood, and `bic` supports component/covariance model selection. Multiple starts reduce local-optimum risk.

9. Training / Evaluation

Standardize features, compare K and covariance types with BIC/AIC and stability, then inspect domain usefulness. With audit labels, use ARI/NMI for clusters or PR-AUC for likelihood-based anomalies; do not assume the highest likelihood model produces meaningful segments.

10. Complexity and Cost

For full covariance, EM is about $O(i \cdot n \cdot K \cdot d^2)$ plus matrix operations and stores $O(K \cdot d^2)$. Diagonal covariance is much cheaper; CPUs are typical.

11. Common Use Cases

Soft customer segments, speaker/background modeling, image segmentation, density-based anomaly detection, missing-data imputation, and generative sampling.

12. Common Mistakes

Using GMM on categorical IDs; not scaling; using too many full-covariance components on little data; reading probabilities as calibrated business probabilities; ignoring singular covariance warnings.

13. Edge Cases / Limitations

Gaussian ellipses cannot model arbitrary shapes efficiently. EM can collapse a covariance around one point, find local optima, and suffer in high dimensions or with heavy outliers.

14. Variations

- **Diagonal GMM:** independent features within each component; scalable baseline.
- **Bayesian GMM:** prior-driven effective component pruning; useful when K is uncertain.
- **Mixture of factor analyzers:** low-rank covariance; research/high-dimensional use.

15. Related Topics

K-Means is a limiting special case with equal spherical covariance and hard assignments. HMMs use GMM-like emissions for sequences. KDE is nonparametric density estimation.

16. Interview Questions

Question	Answer
GMM versus K-Means?	Soft probabilistic ellipses versus hard spherical Voronoi clusters.
What does EM do?	Alternates expected memberships and parameter maximization.
What are responsibilities?	Posterior component probabilities for a point.

Question	Answer
Why BIC?	It trades likelihood against parameter count.
Why can EM fail?	Local optima and degenerate covariances.
Full versus diagonal covariance?	Flexible correlations versus cheaper independence assumption.
Is GMM supervised?	No, unless labels guide an extension.
How score anomalies?	Low $\log p(\mathbf{x})$ indicates poor fit.
Do weights sum to one?	Yes.
Why regularize covariance?	To avoid numerically singular matrices.

17. Practice Tasks

Implement one EM iteration; plot covariance ellipses; compare K-Means/GMM on elongated blobs; choose K with BIC; detect synthetic low-likelihood points.

18. Project Ideas

- **Soft customer personas:** Pandas + GMM; Online Retail; demonstrates probabilistic segmentation.
- **Color image segmenter:** OpenCV + GMM; Berkeley images; shows soft pixel assignment.
- **Sensor density monitor:** scikit-learn + GMM; UCI gas sensors; likelihood-based alerting story.

19. Quick Revision

Key idea: weighted Gaussian density with soft clusters. Formula: $p(\mathbf{x}) = \sum p_i N_k$. Use for elliptical groups/density. Metric: BIC/log likelihood. Trap: covariance overfitting. One-liner: K-Means with probabilities and covariance geometry.

20. Final Cheat Sheet

Definition: finite Gaussian density mixture. Input/output: numeric vectors $\rightarrow$ posterior, label, likelihood. Steps: initialize, E-step, M-step. Hyperparameters: components, covariance type, `reg_covar`. Pros: soft/elliptical; cons: Gaussian/local optima. Best: density and soft clustering.

Association Rule Mining

1. Overview

Association rule mining discovers co-occurrence patterns such as $\{\text{bread, butter}\} \rightarrow \{\text{jam}\}$ in transactional data. It supports basket analysis, recommendation hints, website navigation analysis, and incident co-occurrence exploration.

2. Intuition

Treat each shopping basket as a set. A useful rule says that when a particular set appears, another item appears unusually often relative to its usual frequency.

3. Prerequisites

Sets, transactions, conditional probability, contingency tables, support/confidence, multiple comparisons, sparse binary data.

4. Core Concepts

- **Itemset:** a set of items, e.g. {milk, bread}.
- **Support:** fraction of baskets containing an itemset; filters rare noise.
- **Confidence:** conditional probability of consequent given antecedent.
- **Lift:** dependence relative to chance; greater than one indicates positive association, not causation.

5. Algorithm / Working Process

Input is transactions, each a set of items. Find frequent itemsets above minimum support, create nonempty antecedent/consequent splits, calculate rule metrics, filter redundant/unactionable rules, then validate in time or an A/B experiment. At inference, match a current basket to rules for candidate suggestions.

6. Mathematical Foundation

For rule $A \rightarrow B$, $\text{support}(A) = \text{count}(A)/N$, $\text{support}(A \cup B) = \text{count}(A \cup B)/N$, $\text{confidence}(A \rightarrow B) = \text{support}(A \cup B) / \text{support}(A) = P(B|A)$, and $\text{lift} = \text{confidence}(A \rightarrow B) / \text{support}(B) = P(A \cup B) / (P(A)P(B))$. Leverage is $\text{support}(A \cup B) - \text{support}(A)\text{support}(B)$; conviction is $(1 - \text{support}(B)) / (1 - \text{confidence})$.

7. Practical Implementation

```
# pip install mlxtend
import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

transactions = pd.DataFrame({"bread": [1,1,0,1], "milk": [1,0,1,1], "butter": [1,1,0,0]}).astype(bool)
itemsets = apriori(transactions, min_support=0.25, use_colnames=True)
rules = association_rules(itemsets, metric="lift", min_threshold=1.0)
print(rules[["antecedents", "consequents", "support", "confidence", "lift"]])
```

8. Code Explanation

Rows are transactions and columns are Boolean item indicators. `apriori` produces frequent itemsets; `association_rules` evaluates every valid directional split and filters by lift.

9. Training / Evaluation

Use a time-based holdout because purchase patterns drift. Set support high enough for reliable counts, rank rules with confidence/lift/leverage, deduplicate near-equivalent rules, and test incremental conversion or basket value online.

10. Complexity and Cost

The itemset search is exponential in item count in the worst case. Sparse representations and support thresholds are crucial; rule generation can also explode for large itemsets. CPU is sufficient for ordinary catalog-scale analysis.

11. Common Use Cases

Cross-sell bundles, coupon design, page-path analysis, co-prescribed items, alarm correlation, and feature co-occurrence audits.

12. Common Mistakes

Calling lift causal; accepting high-confidence rules with a very common consequent; using raw transaction IDs as items; ignoring item quantity/time/season; deploying a rule without a controlled outcome test.

13. Edge Cases / Limitations

Rare but valuable associations may be filtered out; huge catalogs cause combinatorial blow-up; rules are descriptive and cannot personalize as well as collaborative filtering.

14. Variations

- **FP-Growth:** compresses transactions in an FP-tree; usually faster than Apriori.
- **Sequential pattern mining:** preserves order/time, e.g. browse then buy.
- **Weighted/utility rules:** incorporate profit or quantity; useful for real retail projects.

15. Related Topics

Apriori is the classic frequent-itemset algorithm. Collaborative filtering learns user-item preferences, while association rules are aggregate co-occurrence. Causal inference is needed for treatment effects.

16. Interview Questions

Question	Answer
What is support?	Fraction of transactions containing an itemset.
What is confidence?	$\frac{P(A \cup B)}{P(B)}$
What is lift?	Observed co-occurrence divided by independence expectation.
Does lift imply causality?	No.
Why can confidence mislead?	A very common consequent gives high confidence.
Why minimum support?	It removes unreliable rare itemsets and reduces search.

Question	Answer
Is $A \rightarrow B$ same as $B \rightarrow A$?	Support is same; confidence usually differs.
What is a transaction?	One set/basket/session of items.
What validates a rule?	Time holdout and ideally an A/B test.
Better algorithm for scale?	FP-Growth often is.

17. Practice Tasks

Calculate support/confidence/lift manually; mine Groceries data; compare rules at three support thresholds; remove redundant rules; design an A/B test for a cross-sell placement.

18. Project Ideas

- **Basket bundle recommender:** Pandas + mlxtend; Online Retail; clear business story.
- **Course-path analyzer:** association rules + Streamlit; MOOC activity data; shows product analytics.
- **Security alert correlator:** PySpark FP-Growth; log event data; enterprise-scale relevance.

19. Quick Revision

Key idea: find useful co-occurrence rules. Formula: $\text{lift} = P(A, B) / (P(A)P(B))$. Use for baskets/sets. Metrics: support, confidence, lift. Trap: correlation is not causation. One-liner: conditional co-occurrence mining.

20. Final Cheat Sheet

Definition: descriptive item co-occurrence rules. Input/output: transactions $\rightarrow A \rightarrow B$ rules. Steps: frequent itemsets, generate splits, filter metrics. Hyperparameters: min support/confidence/lift. Pros: interpretable; cons: combinatorial/noncausal. Best: cross-sell exploration.

Apriori Algorithm

1. Overview

Apriori is a level-wise algorithm for discovering frequent itemsets, commonly followed by association-rule generation. It is a foundational interview topic because its pruning principle explains how to tame combinatorial search.

2. Intuition

If $\{\text{bread, milk, butter}\}$ is frequent, then every subset must be frequent. Conversely, if $\{\text{bread, milk}\}$ is infrequent, never waste work extending it with another item.

3. Prerequisites

Set operations, support, association rules, candidate generation, combinatorics, and transaction encoding.

4. Core Concepts

- **Downward closure / anti-monotonicity:** every subset of a frequent itemset is frequent; key pruning proof.
- **Candidate set C_k :** possible k -itemsets made by joining frequent $(k-1)$ sets.
- **Frequent set L_k :** candidates whose support meets threshold.
- **Pruning:** discard a candidate if any $(k-1)$ subset is absent from $L_{(k-1)}$.

5. Algorithm / Working Process

Scan transactions to obtain frequent singleton set L_1 . Join members of $L_{(k-1)}$ to create C_k , prune candidates with infrequent subsets, scan transactions to count supports, retain L_k , and stop when it is empty. Generate association rules only after all frequent itemsets are known.

6. Mathematical Foundation

The property is $\text{support}(S) \leq \text{support}(T)$ whenever $T \subseteq S$, because every transaction containing S also contains T . Thus $\text{support}(T) < \text{minsup} \Rightarrow \text{support}(S) < \text{minsup}$; an infrequent subset proves every superset infrequent. The candidate count can be $2^m - 1$ for m items, so pruning and thresholding matter.

7. Practical Implementation

```
from mlxtend.frequent_patterns import apriori
import pandas as pd

X = pd.DataFrame({"A": [1, 1, 0, 1], "B": [1, 0, 1, 1], "C": [0, 1, 1, 1]}).astype(bool)
frequent = apriori(X, min_support=0.5, use_colnames=True)
print(frequent.sort_values("support", ascending=False))
```

8. Code Explanation

The library executes candidate generation and support counting. Boolean columns are safer than numeric quantities because Apriori models item presence; preprocess quantities into explicit items if needed.

9. Training / Evaluation

There is no trainable model. Select `min_support` from sample size and business relevance, evaluate resulting rules on later transactions, and use rule metrics plus online impact. Track how candidate count changes with the threshold.

10. Complexity and Cost

Worst-case time and candidate space are exponential; it needs repeated database scans and is slow for dense data with low support. Memory holds candidate itemsets. FP-Growth is typically preferable at scale.

11. Common Use Cases

Educational demonstrations, small-to-medium market basket data, interpretable co-occurrence audits, and candidate-rule generation.

12. Common Mistakes

Confusing Apriori with all association-rule mining; generating rules before mining itemsets; setting support too low; ignoring sparse encoding; saying all subsets of an infrequent itemset are infrequent (the direction is wrong).

13. Edge Cases / Limitations

Low support or many common items can create enormous candidate sets. It misses rare patterns by design and lacks sequence, quantity, user, or causal context.

14. Variations

- **AprioriTid**: uses transaction-ID candidate tables after the first pass.
- **Partition Apriori**: mines partitions then verifies global candidates; distributed-data concept.
- **FP-Growth**: avoids candidate generation; most practical replacement.

15. Related Topics

Association-rule mining consumes Apriori's frequent itemsets. Eclat uses vertical transaction ID lists. FP-Growth uses a compressed prefix tree.

16. Interview Questions

Question	Answer
State Apriori property.	All subsets of a frequent itemset are frequent.
Why does it help?	Infrequent subsets prune all their supersets.
What is C_k ?	Candidate frequent k -itemsets.
What is L_k ?	Candidates surviving minimum support.
Why repeated scans?	Each level needs support counts.
Can low support be costly?	Yes, it creates many candidates.
Does Apriori generate final rules directly?	It mines itemsets first; rules are derived later.
Is support monotonic upward?	No; support cannot increase when items are added.
Apriori versus FP-Growth?	Candidate scans versus compressed tree mining.
What data format?	Transactions as sets or Boolean item columns.

17. Practice Tasks

Hand-run L_1 , C_2 , and L_2 ; implement candidate join/prune for a toy dataset; measure candidates under changing support; compare runtime with FP-Growth.

18. Project Ideas

- **Small-store basket miner:** Python + mlxtend; Groceries dataset; interview-friendly explanation of pruning.
- **Library borrowing patterns:** Pandas + Apriori; public library circulation data; interpretable analytics.
- **Incident co-occurrence audit:** Spark MLlib; IT incident records; shows scalable association discovery.

19. Quick Revision

Key idea: frequent supersets need frequent subsets. Formula: $\text{support}(S) \leq \text{support}(T)$ for $T \subseteq S$. Use for small/teaching basket mining. Metric: support. Trap: property direction. One-liner: level-wise frequent-itemset search with anti-monotone pruning.

20. Final Cheat Sheet

Definition: candidate-and-prune frequent itemset miner. Input/output: transaction matrix $\rightarrow$ frequent itemsets. Steps: scan, join, prune, count, repeat. Hyperparameter: minimum support. Pros: simple/explainable; cons: repeated scans/exponential. Best: foundational and modest datasets.

Matrix Factorization

1. Overview

Matrix factorization approximates a sparse user-item interaction matrix using low-dimensional latent vectors. It powers collaborative filtering, recommendations, representation learning, and some topic-model-like decompositions.

2. Intuition

Instead of memorizing every user rating, describe each user by a few hidden tastes and each item by hidden attributes. Their dot product estimates compatibility: a user who likes "action" dimensions scores action-heavy films highly.

3. Prerequisites

Matrices, dot products, gradients, regularization, sparse data, train/test splits by time, and ranking metrics.

4. Core Concepts

- **Latent factors:** learned dimensions without fixed labels; interpret cautiously.
- **User/item embeddings:** rows of P and Q ; their dot product makes a score.
- **Explicit versus implicit feedback:** ratings versus clicks/purchases; zeros in implicit data usually mean unknown, not dislike.
- **Cold start:** new users/items lack interactions; factorization alone cannot solve it.

5. Algorithm / Working Process

Input is observed interactions (`user`, `item`, `value`). Initialize user matrix `P` and item matrix `Q`; train only on observed entries with SGD or alternating least squares; rank unseen items by $p_u^T q_i$, excluding already seen items. For inference, fetch vectors and score a candidate set.

6. Mathematical Foundation

For observed index set Ω , minimize $\min_{(P,Q)} \sum_{(u,i) \in \Omega} (r_{ui} - p_u^T q_i)^2 + \lambda(\|P\|_F^2 + \|Q\|_F^2)$. With biases: $\hat{r}_{ui} = \mu + b_u + b_i + p_u^T q_i$. SGD uses residual $e_{ui} = r_{ui} - \hat{r}_{ui}$ and updates $p_u \leftarrow p_u + \eta(e_{ui} q_i - \lambda p_u)$, analogously for q_i .

7. Practical Implementation

```
import numpy as np
from sklearn.decomposition import NMF

R = np.array([[5, 4, 0, 0], [4, 0, 0, 2], [0, 2, 4, 5]], dtype=float)
model = NMF(n_components=2, init="nndsvda", max_iter=1000, random_state=42)
P = model.fit_transform(R)          # nonnegative user factors
Q = model.components_               # item factors
print(np.round(P @ Q, 2))
```

8. Code Explanation

NMF is a nonnegative factorization baseline. The reconstructed matrix $P @ Q$ estimates entries; for real explicit-feedback recommendation, avoid treating missing ratings as zero and use a library/implementation that trains on observed interactions only.

9. Training / Evaluation

Use chronological interaction splits; prevent future interactions in features; evaluate ranking with Recall@K, Precision@K, NDCG@K, MAP, coverage, and novelty. Tune factors, regularization, learning rate, and confidence weighting. Check popularity bias and offline-to-online mismatch.

10. Complexity and Cost

Sparse SGD costs roughly $O(|\Omega| * f * \text{epochs})$ and stores $O((\text{users} + \text{items}) * f)$. Candidate ranking can be $O(\text{items} * f)$ per user unless approximate retrieval is used. GPUs are optional for large models.

11. Common Use Cases

Movie/product/music recommendations, job matching, personalized feeds, implicit-feedback ranking, and compressed interaction representations.

12. Common Mistakes

Randomly splitting time data; treating every missing entry as a negative rating; recommending consumed items; reporting RMSE only for a ranking product; ignoring cold start, popularity bias, and feedback loops.

13. Edge Cases / Limitations

Sparse new users/items, evolving tastes, nonstationary catalogs, and side-information needs limit basic MF. Latent factors can reinforce popularity and may not be interpretable.

14. Variations

- **SVD/biased MF:** explicit ratings with global/user/item biases; placement-relevant.
- **ALS implicit MF:** confidence-weighted clicks/purchases; scalable Spark baseline.
- **BPR:** pairwise ranking loss for implicit feedback; important recommender extension.
- **Neural collaborative filtering:** nonlinear interaction; research/project use.

15. Related Topics

SVD is a dense linear-algebra factorization; recommender MF is trained on sparse observations. Autoencoders can reconstruct user vectors. Association rules give nonpersonalized co-occurrence recommendations.

16. Interview Questions

Question	Answer
What does MF learn?	Low-dimensional user and item latent vectors.
Why regularize?	Sparse data makes embeddings easy to overfit.
What is cold start?	No interactions for a new user or item.
Explicit versus implicit?	Ratings versus behavioral signals with uncertainty.
Why time split?	Random split leaks future preference information.
RMSE versus NDCG?	Rating error versus ranked-list quality.
How predict a rating?	Bias terms plus user-item dot product.
Why not zeros for missing ratings?	Missing generally means unobserved, not dislike.
SGD versus ALS?	Joint incremental updates versus alternating closed-form subproblems.
How scale retrieval?	ANN search over item vectors/candidate generation.

17. Practice Tasks

Build biased SGD on MovieLens 100K; compare 10/50/100 factors; evaluate Recall@10 using temporal split; inspect popularity bias; add genre features for cold-start items.

18. Project Ideas

- **Movie recommender:** Surprise/implicit + FastAPI; MovieLens; classic placement project with ranking metrics.
- **Job recommender:** PyTorch + FAISS; job-click data; demonstrates candidate retrieval.

- **Course recommender:** Spark ALS + dashboard; EdNet/online-course events; scalable personalization story.

19. Quick Revision

Key idea: interaction matrix R approximates PQ^T . Formula: regularized observed-entry loss. Use for personalization. Metrics: Recall/NDCG@K. Trap: missing is not negative. One-liner: learn tastes and item traits as vectors.

20. Final Cheat Sheet

Definition: low-rank interaction modeling. Input/output: interactions $\rightarrow$ user/item vectors and rankings. Steps: factorize observed entries, score unseen candidates. Hyperparameters: factors, regularization, epochs. Pros: scalable/personalized; cons: cold start/bias. Best: collaborative recommendation.

Autoencoders

1. Overview

An autoencoder is a neural network trained to reconstruct its input through a constrained latent representation. It supports nonlinear compression, denoising, representation learning, anomaly detection, and generative-model building blocks.

2. Intuition

Force a network to pass an input through a narrow bottleneck. To rebuild the input, it must retain useful structure rather than every raw detail. A denoising version learns to map a noisy image back to a clean one.

3. Prerequisites

Neural networks, tensors, activations, backpropagation, MSE/BCE, regularization, PyTorch, normalization, and train/validation splits.

4. Core Concepts

- **Encoder/latent/decoder:** $z=f_{\theta}(x)$ compresses; $\hat{x}=g_{\phi}(z)$ reconstructs.
- **Bottleneck:** limited latent capacity encourages useful features; an overlarge one can learn identity copying.
- **Reconstruction loss:** measures fidelity; it does not guarantee semantic features.
- **Regularization:** noise, sparsity, or KL constraints prevent trivial copying.

5. Algorithm / Working Process

Input is an unlabeled normalized vector. The encoder produces latent code z , decoder reconstructs $\hat{x}$, loss compares $\hat{x}$ to target x , and backpropagation updates both networks. At inference, use z as features, $\hat{x}$ for reconstruction, or error as an anomaly score after validating its threshold.

6. Mathematical Foundation

Basic objective: $\min_{(\theta, \phi)} \frac{1}{n} \sum_i L(x_i, g_{\phi}(f_{\theta}(x_i)))$. For standardized continuous inputs, $L = \|x - \hat{x}\|_2^2$; for binary pixels, use BCE. Denoising AE trains $L(x, g(f(x_{\text{tilde}})))$ with corrupted x_{tilde} . Sparse AE adds $\lambda \sum_j |z_j|$ (or KL sparsity). A VAE instead optimizes $E_q[\log p(x|z)] - KL(q(z|x)||p(z))$.

7. Practical Implementation

```
import torch
from torch import nn

class Autoencoder(nn.Module):
    def __init__(self, d):
        super().__init__()
        self.encoder = nn.Sequential(nn.Linear(d, 32), nn.ReLU(), nn.Linear(32,
8))
        self.decoder = nn.Sequential(nn.Linear(8, 32), nn.ReLU(), nn.Linear(32,
d))
    def forward(self, x):
        return self.decoder(self.encoder(x))

X = torch.randn(256, 20)
net = Autoencoder(20)
opt = torch.optim.Adam(net.parameters(), lr=1e-3)
for _ in range(100):
    opt.zero_grad(); loss = nn.functional.mse_loss(net(X), X); loss.backward();
    opt.step()
    print(loss.item())
```

8. Code Explanation

The encoder maps 20 features to an 8-value bottleneck and the decoder maps back. The optimizer must receive `net.parameters()` from the exact model being trained; MSE trains reconstruction on unlabeled normalized data.

9. Training / Evaluation

Fit preprocessing on training data only and reserve validation data for architecture, learning-rate, and early-stopping choices. Track validation reconstruction loss and qualitative reconstructions. For anomaly detection, train on verified normal data, choose a validation threshold, and report PR-AUC/recall at review capacity.

10. Complexity and Cost

Per epoch is roughly proportional to samples times layer weights; memory includes activations for backpropagation. Small tabular AEs run on CPU; image/convolutional AEs benefit from GPUs.

11. Common Use Cases

Denoising images/audio, nonlinear feature compression, embedding initialization, missing-value reconstruction, industrial anomaly scoring, and pretraining.

12. Common Mistakes

Training anomalies into an anomaly AE; using unnormalized features with MSE; bottleneck too large; interpreting low reconstruction error as semantic understanding; evaluating only training loss; optimizer tied to a different model instance.

13. Edge Cases / Limitations

A powerful AE may reconstruct anomalies well, especially if contaminated training data contains them. Reconstruction error can focus on high-variance dimensions. Latent coordinates are non-identifiable and plain AEs are not generative samplers.

14. Variations

- **Denoising AE:** reconstruct clean input from corruption; practical robustness.
- **Convolutional AE:** spatial encoder/decoder for images; project-relevant.
- **Variational AE:** probabilistic latent space/generation; research/GenAI important.
- **Sparse/contractive AE:** explicit representation constraints; conceptual interview extension.

15. Related Topics

PCA is a linear AE under restricted conditions. VAEs add a distributional latent objective. Matrix factorization is low-rank reconstruction for a matrix; Transformers can use masked reconstruction pretraining.

16. Interview Questions

Question	Answer
What is an AE's target?	Usually its own input.
Why bottleneck?	It limits capacity and encourages compact structure.
AE versus PCA?	Nonlinear neural reconstruction versus linear orthogonal projection.
Which loss for images?	MSE for continuous normalized pixels, BCE for Bernoulli-like pixels.
Can an AE detect anomalies?	Yes via high reconstruction error, with clean normal training.
Why might it fail at anomalies?	High-capacity models can reconstruct them too.
What is denoising AE?	Reconstruct clean inputs from corrupted versions.
Is a plain AE generative?	No; it has no guaranteed latent prior.
How avoid overfitting?	Validation, bottleneck, noise, regularization, early stopping.
What adds probabilistic generation?	A VAE's latent distribution and KL term.

17. Practice Tasks

Train a tabular AE with early stopping; compare latent sizes; visualize reconstructions; add Gaussian noise; train only normals and calibrate anomaly threshold; compare with PCA.

18. Project Ideas

- **Industrial fault detector:** PyTorch + FastAPI; MVTec AD/sensor data; strong anomaly pipeline story.
- **Image denoiser:** PyTorch CNN AE; CIFAR-10; demonstrates visual reconstructions.
- **Compression study:** PyTorch + PCA; Fashion-MNIST; compares linear/nonlinear latent spaces.

19. Quick Revision

Key idea: reconstruct input through a latent bottleneck. Formula: $\min_x L(x, g(f(x)))$. Use for nonlinear representation/denoising. Metric: validation reconstruction and task metric. Trap: identity copying/contaminated normals. One-liner: neural nonlinear compression by self-reconstruction.

20. Final Cheat Sheet

Definition: encoder-decoder reconstruction network. Input/output: $x \rightarrow z \rightarrow \hat{x}$. Steps: encode, decode, backpropagate loss. Hyperparameters: latent size, layers, loss, LR. Pros: nonlinear/flexible; cons: needs tuning/no semantic guarantee. Best: compression, denoising, normal-pattern modeling.

Self-Organizing Maps (SOM)

1. Overview

A self-organizing map is an unsupervised neural grid whose prototype vectors learn to represent data while preserving neighborhood topology. It is useful for interpretable 2-D maps of high-dimensional profiles, especially in exploratory analytics.

2. Intuition

Place many small reference points on a 2-D rubber sheet. For each data point, move the closest reference point toward it and tug its nearby grid neighbors too. Similar inputs settle in nearby map cells.

3. Prerequisites

Vectors, Euclidean/cosine distance, competitive learning, grid topology, learning-rate schedules, and clustering basics.

4. Core Concepts

- **Neuron/prototype:** grid cell with weight vector w_j in input space.
- **BMU:** best matching unit, the nearest prototype to input x .
- **Neighborhood function:** nearby grid cells also update, preserving topology.
- **U-matrix:** visualizes distances between neighboring prototypes; high borders suggest cluster boundaries.

5. Algorithm / Working Process

Initialize a 2-D grid of prototype vectors. For each input, find its BMU, move BMU and neighboring nodes toward the input, then shrink learning rate and neighborhood radius over epochs. Output is a BMU coordinate per row and an interpretable prototype map; new points map to their nearest learned prototype.

6. Mathematical Foundation

$c = \text{argmin}_j ||x - w_j||$. Update each node: $w_j(t+1) = w_j(t) + \alpha(t) h_{cj}(t)[x - w_j(t)]$, where a common Gaussian neighborhood is $h_{cj}(t) = \exp(-||r_c - r_j||^2 / (2 \sigma(t)^2))$. Both $\alpha(t)$ and $\sigma(t)$ decay, shifting from global ordering to local refinement. Quantization error is mean $||x - w_c||$.

7. Practical Implementation

```
# pip install minisom
from minisom import MiniSom
from sklearn.datasets import load_wine
from sklearn.preprocessing import StandardScaler

X, _ = load_wine(return_X_y=True)
X = StandardScaler().fit_transform(X)
som = MiniSom(10, 10, X.shape[1], sigma=1.5, learning_rate=0.5, random_seed=42)
som.random_weights_init(X); som.train_random(X, 1000)
print(som.winner(X[0]), som.quantization_error(X))
```

8. Code Explanation

The 10×10 map has 100 prototypes. Scaling is essential because BMU distance is scale-sensitive. `winner` maps a point to a grid coordinate; quantization error measures prototype fit.

9. Training / Evaluation

Scale and sample representative data. Tune map dimensions, initial radius, learning rate, and epochs; inspect quantization/topographic error, U-matrix, seed stability, and downstream usefulness. There is no train/validation label target, but held-out quantization error can catch overfitting/poor coverage.

10. Complexity and Cost

Naive training costs $O(\text{epochs} * n * m * d)$ for m map nodes, though neighborhood updates can be optimized. Memory is $O(m * d)$. CPU is usually enough.

11. Common Use Cases

Customer/persona maps, telecom/network profiles, gene-expression maps, sensor operating modes, and visual exploratory dashboards.

12. Common Mistakes

Skipping scaling; equating every grid cell with a cluster; using too large a map for little data; not decaying radius; claiming SOM is a deep neural classifier; reading 2-D map distance as exact original-space distance.

13. Edge Cases / Limitations

SOM is sensitive to grid size, schedule, and initialization; it is less scalable and less principled than modern UMAP for many visual tasks. Topology can fold and does not guarantee cluster boundaries.

14. Variations

- **Batch SOM:** update from batch statistics; stable/scalable option.
- **Growing SOM:** adds nodes adaptively; research/variable-complexity use.
- **Toroidal SOM:** wraps grid boundaries; reduces edge artifacts.

15. Related Topics

K-Means has prototypes but no neighborhood grid. UMAP/t-SNE create maps primarily for visualization. Vector quantization uses nearest codebook entries without topological organization.

16. Interview Questions

Question	Answer
What is a BMU?	The prototype closest to an input.
Why update neighbors?	To preserve grid topology.
What decays during training?	Learning rate and neighborhood radius.
SOM versus K-Means?	Both use prototypes; SOM also imposes a 2-D neighborhood.
What is a U-matrix?	Neighbor-prototype distance visualization.
Is SOM supervised?	No.
Why scale inputs?	BMU selection uses distance.
What is quantization error?	Distance from inputs to BMUs.
How map new data?	Find its closest learned prototype.
Main limitation?	Hyperparameter-sensitive visualization, not exact geometry.

17. Practice Tasks

Train maps of 5x5/10x10/20x20; plot a U-matrix; label BMUs using known labels only for audit; compare quantization error; use cosine-normalized text vectors.

18. Project Ideas

- **Customer persona map:** MiniSom + Plotly; Online Retail; highly visual segmentation portfolio piece.
- **Wine profile explorer:** scikit-learn + SOM; UCI Wine; interview demo of BMUs/U-matrix.
- **Network state monitor:** MiniSom + alerts; KDD Cup/telemetry data; operations visualization story.

19. Quick Revision

Key idea: prototypes on a topology-preserving grid. Formula: neighbor-weighted prototype update. Use for interpretable 2-D profile maps. Metric: quantization/topographic error. Trap: calling cells true clusters. One-liner: competitive learning plus neighborhood cooperation.

20. Final Cheat Sheet

Definition: topology-preserving prototype map. Input/output: vectors $\rightarrow$ BMU grid cells. Steps: find BMU, update neighbors, decay schedules. Hyperparameters: map size, radius, LR, epochs. Pros: interpretable map; cons: sensitive/slower. Best: exploratory profile visualization.

Spectral Clustering

1. Overview

Spectral clustering groups data by constructing a similarity graph and clustering its Laplacian eigenvector embedding. It can recover non-convex groups such as moons when a meaningful local graph can be built.

2. Intuition

Imagine points linked by strong rubber bands to similar neighbors. Cutting the graph into groups requires cutting few strong links. Eigenvectors reveal a low-dimensional representation where these weakly connected regions separate.

3. Prerequisites

Graphs, adjacency matrices, degree matrices, eigenvalues/eigenvectors, Laplacian, K-Means, similarity kernels, and sparse matrices.

4. Core Concepts

- **Affinity matrix W** : pairwise similarities; graph quality dominates final quality.
- **Graph Laplacian**: $L = D - W$; encodes smoothness over graph edges.
- **Eigenvectors**: smallest Laplacian eigenvectors embed nodes so connected nodes remain close.
- **Normalized cut**: objective balancing edge cuts against cluster volume; motivates normalized Laplacians.

5. Algorithm / Working Process

Input is samples and an affinity definition (RBF, nearest-neighbor, or precomputed graph). Build W , compute degree D , form a normalized Laplacian, take the first k informative eigenvectors, row-normalize if required, then apply K-Means to those rows. Output is graph-partition labels; new-point assignment is not naturally supported.

6. Mathematical Foundation

$W_{ij} = \exp(-||x_i - x_j||^2 / (2 \sigma^2))$ or kNN connectivity. $D_{ii} = \sum_j W_{ij}$. Unnormalized Laplacian $L = D - W$; symmetric normalized $L_{sym} = I - D^{-1/2} W D^{-1/2}$. The relaxed normalized-cut problem uses the k

smallest eigenvectors of L_{sym} , then K-Means clusters their rows. For any vector f , $f^T L f = \frac{1}{2} \sum_{ij} w_{ij}(f_i - f_j)^2$, so strongly connected nodes prefer similar embedding values.

7. Practical Implementation

```
from sklearn.datasets import make_moons
from sklearn.cluster import SpectralClustering

X, _ = make_moons(n_samples=400, noise=0.06, random_state=42)
model = SpectralClustering(n_clusters=2, affinity="nearest_neighbors",
                           n_neighbors=12, assign_labels="kmeans",
                           random_state=42)
labels = model.fit_predict(X)
print(labels[:10])
```

8. Code Explanation

A kNN affinity avoids selecting an RBF bandwidth. Spectral clustering finds the graph embedding internally, then K-Means assigns labels. Plot X colored by `labels` to see the non-convex split.

9. Training / Evaluation

Scale numerical features, select graph type/`n_neighbors` or RBF `gamma`, and check connected components. Evaluate silhouette in an appropriate space cautiously, cluster stability, and ARI/NMI only with withheld audit labels. Tune `k` with eigengaps plus domain needs.

10. Complexity and Cost

Dense affinity needs $O(n^2)$ memory and eigendecomposition can reach $O(n^3)$, making vanilla spectral clustering unsuitable for large n . Sparse kNN graphs and iterative eigensolvers reduce cost but remain heavier than K-Means.

11. Common Use Cases

Two-moons-style shapes, image segmentation, community detection, graph partitioning, and small-to-medium nonlinear cluster analysis.

12. Common Mistakes

Using a dense RBF graph on huge data; choosing similarity bandwidth blindly; ignoring disconnected graphs; assuming it works automatically in high-dimensional noisy data; using it as a production out-of-sample classifier.

13. Edge Cases / Limitations

Sensitive affinity construction can bridge separate groups or split one group. Scaling is difficult, eigendecomposition is costly, and imbalanced/variable-density data can be troublesome.

14. Variations

- **Normalized spectral clustering:** balances cluster sizes/volumes; standard placement topic.
- **Nyström approximation:** approximates eigenvectors for larger data.
- **Graph community methods:** Louvain/Leiden for large network graphs rather than feature-space RBF graphs.

15. Related Topics

DBSCAN handles density-connected arbitrary shapes without fixing k . Kernel PCA also uses eigenvectors of similarity structure. Graph neural networks learn from graphs but are supervised/self-supervised neural models.

16. Interview Questions

Question	Answer
Why can it find moons?	The graph encodes local connectivity rather than centroid geometry.
What is W ?	Pairwise affinity/adjacency matrix.
Define L .	Usually degree minus affinity: $D - W$.
Why eigenvectors?	They relax graph-cut objectives into a solvable embedding.
What follows eigenvectors?	K-Means on row embeddings.
Why normalize Laplacian?	To account for node degree/cluster volume.
Main bottleneck?	Building graph and eigendecomposition.
RBF gamma effect?	It controls similarity locality.
Does it support new samples?	Not naturally in standard formulation.
Spectral versus DBSCAN?	Graph cut with fixed k versus density-connected clusters/noise.

17. Practice Tasks

Compare K-Means/DBSCAN/spectral on moons; sweep kNN neighbors and RBF gamma; inspect graph components/eigengap; use sparse affinity; segment a small image.

18. Project Ideas

- **Image segmenter:** scikit-image + spectral clustering; Berkeley images; classic CV graph project.
- **Community explorer:** NetworkX + spectral embedding; Karate Club/social network; graph-ML portfolio story.
- **Shape-aware customer clusters:** scikit-learn + dashboard; synthetic/behavior embeddings; demonstrates method selection.

19. Quick Revision

Key idea: cluster eigenvectors of a similarity graph. Formula: $L=D-W$. Use for small non-convex groups. Metric: stability/ARI audit. Trap: $O(n^2)$ graph cost. One-liner: a relaxed graph-cut followed by K-Means.

20. Final Cheat Sheet

Definition: graph-Laplacian eigenvector clustering. Input/output: samples/affinity -> labels. Steps: graph, Laplacian, eigenvectors, K-Means. Hyperparameters: clusters, affinity, neighbors/gamma. Pros: non-convex shapes; cons: graph-sensitive/costly. Best: small graph-like cluster problems.

Isolation Forest

1. Overview

Isolation Forest is an efficient tree-ensemble anomaly detector. It is one of the best classical baselines for multivariate tabular outliers because it isolates unusual points rather than estimating every normal-data region or pairwise distance.

2. Intuition

Repeatedly choose a random feature and random split value. A far-away or rare point is separated from other observations in few splits, producing a short path from a tree root to an external node. Normal points need more random cuts.

3. Prerequisites

Binary trees, random sampling, path length, expected value, anomaly scoring, contamination thresholds, feature preprocessing, and imbalanced evaluation.

4. Core Concepts

- **Isolation tree (iTree):** a randomly split tree built without labels or impurity minimization.
- **Path length $h(x)$:** edges traversed until point x is isolated; shorter implies more anomalous.
- **Subsampling:** each tree trains on psi rows; it improves speed/diversity and makes path normalization defined.
- **Contamination:** used to convert scores into an alert cutoff; it does not alter the core random partition mechanism.
- **Axis-aligned split:** split $x_j < p$ using randomly selected feature j and uniformly selected p between current min/max.

5. Algorithm / Working Process

Input is a numeric matrix, optionally containing a small fraction of anomalies. For each of T trees, sample psi rows, recursively choose a random feature and random split until a point is isolated or height limit is reached. Score a row by mean path length over trees. At inference, pass the row down each fixed tree, average path length, normalize it, and threshold/rank the resulting anomaly score.

6. Mathematical Foundation

For sample size n , unsuccessful binary-search-tree path correction is $c(n)=2H_{n-1}-2(n-1)/n$, with harmonic number $H_m=\sum_{i=1}^m 1/i$ (approximately $\ln(m)+\gamma$). A leaf containing $n_{\text{leaf}} > 1$ adds $c(n_{\text{leaf}})$ to its observed path. For average path $E[h(x)]$ over trees trained on ψ points, the original score is $s(x)=2^{(-E[h(x)]/c(\psi))}$. Thus $s(x)$ near 1 means easy isolation; near 0.5 is ordinary. Libraries may expose a sign/offset-transformed decision score, so verify the API: in scikit-learn, lower `decision_function` means more anomalous.

7. Practical Implementation

```
import numpy as np
from sklearn.ensemble import IsolationForest
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import RobustScaler

rng = np.random.default_rng(7)
X_train = rng.normal(size=(1000, 4)) # mostly normal reference period
X_eval = np.r_[rng.normal(size=(200, 4)), [[8, 8, 8, 8], [-7, 0, 6, 9]]]
model = make_pipeline(RobustScaler(), IsolationForest(
    n_estimators=300, max_samples=256, contamination="auto", random_state=42,
    n_jobs=-1))
model.fit(X_train)
raw = model.decision_function(X_eval) # smaller = more suspicious in
sklearn
top_alerts = np.argsort(raw)[:10]
print(top_alerts, raw[top_alerts])
```

8. Code Explanation

Fit on a historical reference period rather than mixing in future data. `RobustScaler` is often sensible when units differ, though monotonic scaling does not change a single feature's split order; it makes pipeline behavior safer alongside other detectors. `max_samples=256` is the common efficient default. Do not blindly trust `contamination`: rank `raw`, then set a validation/business threshold for the available review queue.

9. Training / Evaluation

Use a time-based normal/reference window and separate later validation period. Engineer contextual features (amount relative to user history, hour, velocity) and prevent future aggregates. If labels exist, report PR-AUC, precision@K, recall@K, false-positive rate at review capacity, alert latency, and expected cost. Tune `n_estimators` until ranking stability plateaus, `max_samples` for runtime/diversity, `max_features` when many noisy dimensions exist, and threshold separately. Monitor score distributions, alert-rate drift, feature missingness, and reviewer-confirmed incidents.

10. Complexity and Cost

An iTree on ψ subsampled rows has expected depth $O(\log \psi)$. Fitting is approximately $O(T * \psi * \log \psi)$ after sampling; scoring n rows is $O(T * n * \log \psi)$. Storage is $O(T * \psi)$ tree nodes in the common case. It parallelizes across CPU cores and usually needs no GPU; feature preprocessing and review operations often cost more than the model.

11. Common Use Cases

Card-payment screening, account takeover signals, network intrusion features, IoT sensor faults, unusual API behavior, data-pipeline outliers, and rare manufacturing operating conditions.

12. Common Mistakes

- Treating output **-1** as a confirmed incident rather than a thresholded alert.
- Using accuracy or ROC-AUC alone for extreme imbalance; report precision at operational **K**.
- Randomly splitting event streams and leaking future behavior aggregates.
- Feeding raw categorical IDs or timestamps as numeric magnitudes; encode meaningful behavior instead.
- Setting contamination to the presumed fraud rate and assuming it calibrates probability.
- Training on a period with many unaddressed incidents, then expecting clean-normal behavior.
- Ignoring clustered anomalies: a dense anomalous campaign can look normal relative to itself.

13. Edge Cases / Limitations

Isolation Forest detects easy-to-isolate observations, not every harmful event. It may miss local anomalies inside a dense region, contextual anomalies without context features, and collective/sequential anomalies. High-dimensional irrelevant features dilute random splits; distribution shift changes normal path lengths. It is less interpretable than a small rule model, and its scores are not calibrated probabilities.

14. Variations

- **Extended Isolation Forest:** uses random hyperplane splits rather than axis-aligned splits; useful for rotated/correlated structure, research-aware extension.
- **SCiForest:** selects splits to increase separation; research variant.
- **Streaming isolation methods:** update for evolving data; use when concept drift and volume require it.
- **Isolation-based feature engineering + supervised model:** retain IF score as one feature once labels mature; practical production progression.

15. Related Topics

LOF flags low local density but needs neighbor searches. One-Class SVM learns a boundary around normal data but can be costly and scale-sensitive. GMM uses low likelihood; autoencoders use high reconstruction error. Random Forest is different: it uses labeled targets and impurity-driven splits, while Isolation Forest uses random splits without labels.

16. Interview Questions

Question	Answer
What principle does Isolation Forest use?	Anomalies are isolated by random partitions in short paths.
Is it supervised?	No; it needs no labels to fit.
What is an iTree split?	Random feature plus random split value within its observed range.

Question	Answer
Why subsample?	Faster, diverse trees and effective anomaly isolation.
What does a short path mean?	The point is easier to isolate and more anomalous.
Give the score form.	$s(x) = 2^{(-E[h(x)]/c(\psi))}$.
What is $c(\psi)$?	Expected path length of a random binary search tree of sample size ψ .
Does contamination estimate probability?	No; it mainly chooses a cutoff in implementations.
Why is it fast?	It avoids pairwise distances and deep full-data trees.
Why might it miss fraud?	Fraud can be locally dense or contextually unusual only.
How evaluate it with rare labels?	PR-AUC and precision/recall at alert budget on a future period.
IF versus One-Class SVM?	IF scales better for tabular data; OCSVM learns a kernel boundary and is more sensitive to scaling.

17. Practice Tasks

Generate global and local outliers; compare path/decision scores; vary `max_samples` 64/256/512 and plot ranking stability; add 100 noise features; perform a time-based fraud-like split; choose threshold at a fixed 1% alert budget; compare IF with LOF and OCSVM.

18. Project Ideas

- **Transaction-risk triage:** scikit-learn + FastAPI + MLflow; Kaggle credit-card fraud; demonstrates time-aware metrics and alert thresholds.
- **IoT predictive-maintenance monitor:** Pandas + Isolation Forest + Grafana; NASA turbofan/sensor data; shows drift dashboard and operations reasoning.
- **API abuse detector:** feature pipeline + IF + Streamlit; synthetic web logs/CIC-IDS; strong AI-engineering observability project.

19. Quick Revision

Key idea: anomalies need fewer random cuts to isolate. Formula: $s(x) = 2^{(-E[h]/c(\psi))}$. Use for scalable multivariate tabular anomaly ranking. Metrics: PR-AUC, precision@K, recall@K. Trap: `contamination` is not fraud probability. One-liner: an unlabeled forest where short average path means unusual.

20. Final Cheat Sheet

Definition: random-partition ensemble anomaly detector. Input/output: numeric rows -> path-based score/alert. Steps: subsample, random splits, average path, threshold. Hyperparameters: trees, `max_samples`, `max_features`, contamination. Pros: fast/scalable/no distribution assumption; cons: contextual/local anomalies and uncalibrated scores. Best: first strong baseline for tabular anomaly detection.

One-Class SVM

1. Overview

One-Class SVM (OCSVM) learns a boundary enclosing most normal observations and labels points outside it as anomalous. It is useful for small-to-medium, clean-normal datasets with potentially nonlinear boundaries, but requires careful scaling and parameter tuning.

2. Intuition

Map normal points into a high-dimensional feature space and draw the widest boundary that separates them from the origin. A new point outside the learned normal region is flagged.

3. Prerequisites

SVM margins, kernels, dot products, RBF kernel, convex optimization, feature scaling, novelty detection, and class imbalance metrics.

4. Core Concepts

- **Kernel:** implicit feature mapping; RBF captures nonlinear normal regions.
- **nu:** upper bound on training outlier fraction and lower bound on support-vector fraction (under standard conditions).
- **gamma:** RBF locality; large gamma creates a wiggly tight boundary, small gamma a smooth broad one.
- **Support vectors:** points that define the boundary; many indicate a complex/tight fit.

5. Algorithm / Working Process

Fit preferably on a clean normal training set. Standardize features, choose kernel and **nu/gamma**, solve the one-class optimization, and obtain **decision_function** values. At inference, negative decision values are outside the learned region; choose a business threshold from a validation period rather than relying only on zero.

6. Mathematical Foundation

Primal OCSVM solves $\min_{(w, \rho, \xi)} \frac{1}{2} \|w\|^2 + \frac{1}{(\nu n)} \sum_i \xi_i - \rho$, subject to $w^T \phi(x_i) \geq \rho - \xi_i, \xi_i \geq 0$. It seeks a maximum-margin separator of data from the origin in feature space. With RBF kernel, $K(x, x') = \exp(-\gamma \|x - x'\|^2)$. Decision is $f(x) = \sum_i \alpha_i K(x_i, x) - \rho$; negative values are outside the learned support.

7. Practical Implementation

```
import numpy as np
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import OneClassSVM

rng = np.random.default_rng(42)
```

```
X_normal = rng.normal(size=(400, 2))
X_eval = np.r_[rng.normal(size=(100, 2)), [[6, 6], [-5, 4]]]
model = make_pipeline(StandardScaler(), OneClassSVM(kernel="rbf", nu=0.03,
gamma="scale"))
model.fit(X_normal)
score = model.decision_function(X_eval) # lower means less normal
print(np.argsort(score)[:5], model.predict(X_eval)[-2:])
```

8. Code Explanation

Only `X_normal` is used to fit a novelty detector. Scaling is mandatory for distance-based RBF kernels. `nu=0.03` expresses a small tolerated boundary/outlier fraction; `gamma="scale"` is a reasonable starting point, not final tuning.

9. Training / Evaluation

Train on verified normal historical data where possible. Hold out future normal and labeled incident data; tune `nu` and `gamma` on PR-AUC/precision@K and alert cost. Examine support-vector fraction, score histogram, sensitivity to scaling, and threshold drift. Use approximate alternatives for large `n`.

10. Complexity and Cost

Kernel methods commonly need $O(n^2)$ memory/time to construct and optimize a kernel matrix, and prediction costs about $O(\#SV * d)$ plus kernel work per point. CPU is typical but OCSVM becomes impractical at large scale.

11. Common Use Cases

Machine-condition normality, quality-control measurements, small security datasets, novelty detection in scientific instruments, and image/text embeddings after dimensionality reduction.

12. Common Mistakes

Not scaling; fitting contaminated data as if it were clean normal; choosing gamma from a visualization; treating `nu` as exact anomaly prevalence; training millions of samples; using a random temporal split; presenting boundary labels as calibrated risk probabilities.

13. Edge Cases / Limitations

It is sensitive to feature scale, `gamma`, `nu`, and contaminated normals. High dimensionality and large samples make kernels costly; multimodal normal data may require careful kernels/parameters. Drift causes boundary staleness.

14. Variations

- **Linear One-Class SVM/SGD variants:** scalable when normal boundary is roughly linear.
- **Support Vector Data Description (SVDD):** minimum enclosing hypersphere formulation; closely related interview topic.
- **Deep SVDD:** neural representation plus compact normal sphere; research/deep anomaly use.

- **Robust covariance/EllipticEnvelope:** Gaussian-elliptical alternative for low-dimensional clean normals.

15. Related Topics

Isolation Forest is usually more scalable for tabular data. LOF is local-density based. Binary SVM learns a two-class separator when verified anomalies exist. Autoencoders learn normal reconstruction rather than a kernel boundary.

16. Interview Questions

Question	Answer
What does OCSVM learn?	A boundary/support region around normal data.
Why is scaling crucial?	RBF kernel distances are unit-sensitive.
What does <code>nu</code> control?	An upper outlier and lower support-vector fraction bound.
What does gamma control?	RBF boundary locality/complexity.
Give RBF kernel.	$\exp(-\gamma \cdot \text{distance})$
What sign signals anomaly?	Negative decision function in the usual formulation.
OCSVM versus binary SVM?	One-class uses normals only; binary uses both labels.
Why is it expensive?	Kernel matrix scales roughly quadratically in samples.
When prefer Isolation Forest?	Large tabular datasets or uncertain boundary geometry.
Can it handle drift?	Only with monitoring and refitting/adaptation.

17. Practice Tasks

Plot RBF decision contours; sweep `nu` and `gamma`; inject 1%, 5%, 10% contamination into normal training; compare raw versus scaled inputs; benchmark OCSVM and IF as `n` grows.

18. Project Ideas

- **Machine normality guard:** scikit-learn + dashboard; UCI SECOM; showcases clean-normal novelty workflow.
- **Embedding novelty filter:** sentence-transformers + PCA + OCSVM; support tickets; demonstrates semantic anomaly screening.
- **Lab-quality monitor:** Pandas + OCSVM; sensor/QC data; strong threshold-calibration discussion.

19. Quick Revision

Key idea: learn a kernel boundary around normal support. Formula: $\min \frac{1}{2} \|w\|^2 + \sum \xi_i / (n - \rho)$. Use for small, clean-normal nonlinear data. Metrics: PR-AUC/precision@K. Trap: scaling and quadratic cost. One-liner: an SVM that separates normal data from the origin.

20. Final Cheat Sheet

Definition: kernel novelty detector. Input/output: normal training vectors -> decision score/alert. Steps: scale, fit boundary, score, threshold. Hyperparameters: kernel, ν , gamma. Pros: flexible boundary; cons: sensitive and unscalable. Best: carefully curated small-to-medium normal datasets.